

Documentación Técnica y Arquitectónica

App Móvil: Farmacias de Turno

Alumno

15 de julio de 2026

Índice

1. Introducción	2
2. Acuerdos Arquitectónicos (Fase 0)	2
2.1. Metodología de Trabajo	2
2.2. Estandarización de Código	2
2.3. Restricciones de Infraestructura y Servicios	2
3. Diseño del Sistema	3
3.1. Diagrama de Contexto (Nivel 1)	3
3.2. Diagrama de Contenedores (Nivel 2)	3

1. Introducción

Este documento tiene como objetivo registrar de manera formal y evolutiva todas las decisiones arquitectónicas, diagramas y configuraciones de la aplicación móvil solicitada por el profesor Sebastián Salazar Molina.

La aplicación está desarrollada en **Flutter** (Dart), orientada principalmente al sistema operativo Android, e implementa integraciones con hardware (GPS), servicios en la nube (Firebase Google Sign-In) y APIs RESTful (para obtención de clima y farmacias de turno).

2. Acuerdos Arquitectónicos (Fase 0)

2.1. Metodología de Trabajo

Para evitar deuda técnica, el desarrollo se lleva a cabo mediante una metodología estrictamente iterativa. Cada componente debe ser diseñado, implementado mediante programación progresiva y finalmente validado mediante revisión de código antes de avanzar a la siguiente etapa.

2.2. Estandarización de Código

Se han definido las siguientes normas de calidad para el código fuente:

- **SDK Android:** `minSdkVersion 21` (Android 5.0) para asegurar compatibilidad con Firebase y Geolocator.
- **Application ID:** `ezekim.farmaciasgps` establecido como identificador único de Android.
- **Gestión de Estado:** Riverpod.
- **Arquitectura:** Feature-First (Agrupación por funcionalidades).
- **Nomenclatura:** `snake_case` para archivos, `PascalCase` para Clases/Enums, y `camelCase` para variables y funciones.
- **Documentación:** Código 100 % en idioma español con documentación activa para propósitos académicos.

2.3. Restricciones de Infraestructura y Servicios

Para garantizar la viabilidad académica del proyecto y evitar costos imprevistos, se ha establecido la directriz estricta de utilizar exclusivamente servicios en la nube y herramientas **100 % gratuitas (Cloud Free)**. Está terminantemente prohibido incorporar dependencias o APIs que requieran el ingreso de tarjetas de crédito o configuración de facturación. Un ejemplo de la aplicación de esta regla es el uso de OpenStreetMap en lugar del servicio comercial de Google Maps API.

3. Diseño del Sistema

3.1. Diagrama de Contexto (Nivel 1)

El diagrama de contexto ilustra cómo el usuario interactúa con la aplicación móvil y los sistemas externos necesarios.

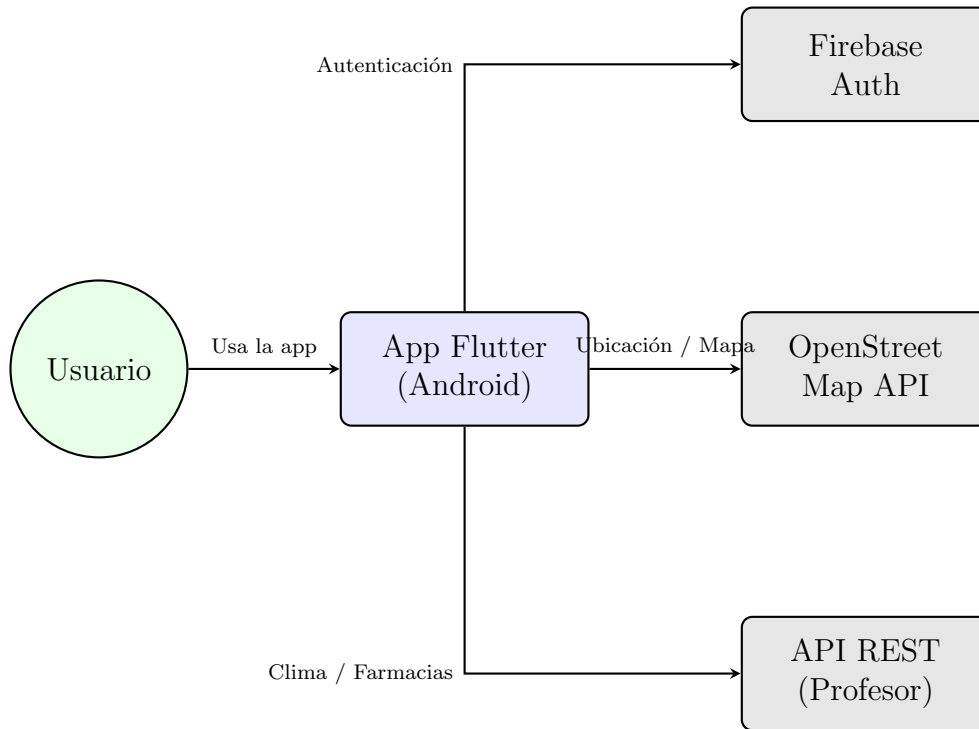


Figura 1: Diagrama de Contexto de la Aplicación.

3.2. Diagrama de Contenedores (Nivel 2)

El diagrama de contenedores detalla la estructura interna de la aplicación Flutter (separación entre UI, Riverpod y Servicios).

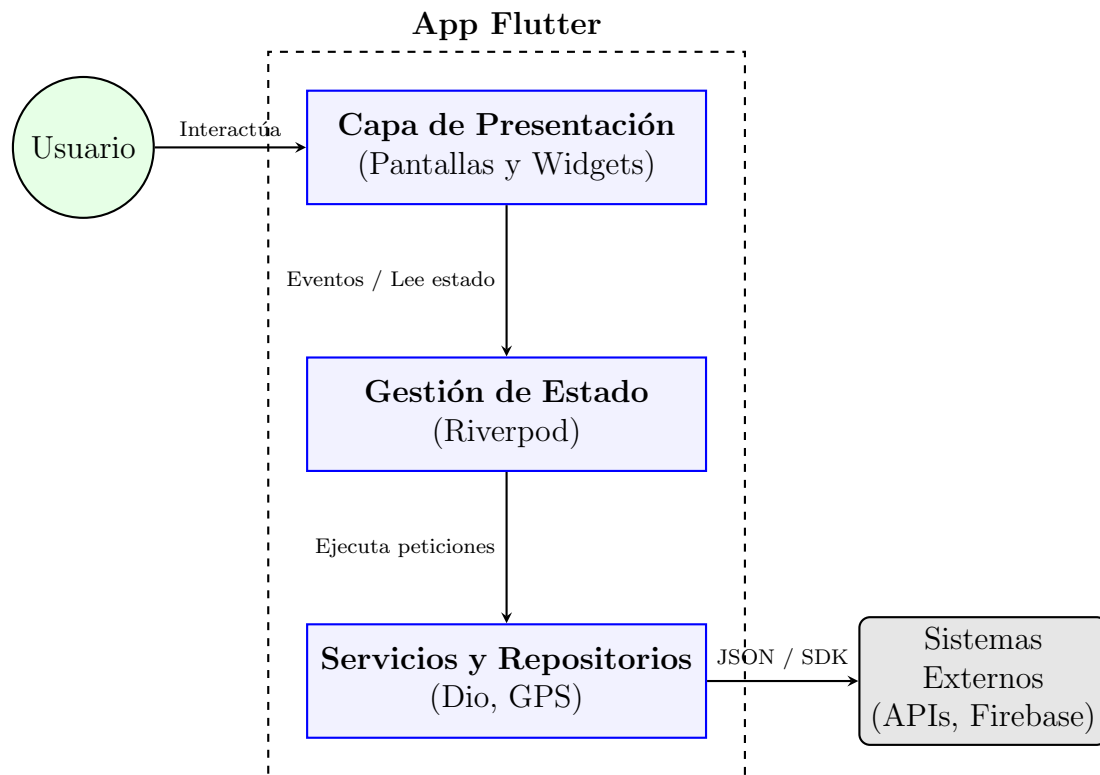


Figura 2: Diagrama de Contenedores Internos.

Explicación del Flujo de Datos

Para comprender mejor la arquitectura de software planteada, es fundamental entender el rol de cada capa y cómo fluye la información desde el usuario hasta los servidores externos:

1. **Capa de Presentación (UI/UX):** Es la interfaz visual pura (botones, mapas, textos). Esta capa es "tonta", es decir, no contiene lógica de negocio ni hace peticiones a internet. Solo captura las acciones del usuario (ej. presionar "Buscar Farmacias") y muestra los datos que le entrega la capa inferior.
2. **Gestión de Estado (Riverpod):** Funciona como el cerebro.º puente intermedio. Cuando el usuario hace una acción en la UI, la Gestión de Estado se entera y decide qué hacer. Su trabajo principal es solicitar datos a los Servicios, mantener esos datos almacenados temporalmente en la memoria del teléfono, y avisarle a la UI de forma reactiva para que se actualice o redibuje cuando los datos cambien.
3. **Servicios y Repositorios:** Son los .ºbreros". Tienen el código específico para comunicarse con el mundo exterior. Utilizan herramientas como **Dio** para hacer peticiones HTTP a las APIs, o paquetes como **Geolocator** para obtener coordenadas del chip GPS.
4. **Sistemas Externos:** Son los proveedores finales de la información que consume la capa de Servicios. Incluyen:
 - **Firebase Auth:** Mantiene la persistencia de sesión segura usando las cuentas de Google del usuario.

- **OpenStreetMap API:** Brinda la cartografía de uso libre y gratuito para dibujar los mapas, evitando los costos de facturación de Google Maps.
- **API del Profesor:** Servidor RESTful que retorna la información climática y la lista de farmacias de turno en bruto (formato JSON).

Ejemplo de un Flujo Completo (paso a paso):

1. El usuario abre la app y pulsa el botón "Ver farmacias cercanas.^{en} la **UI**.
2. Ese botón no hace la búsqueda directa, sino que le avisa a **Riverpod** (Gestión de Estado) que se necesitan los datos.
3. Riverpod llama al **Servicio** de Farmacias y al Servicio de GPS.
4. Los Servicios hacen el "trabajo pesado": Encienden el GPS del teléfono y hacen la petición HTTP a la **API del profesor**.
5. La API responde con un texto en formato JSON. El Servicio toma ese texto, lo "traducez lo convierte en una lista de objetos **Farmacia** que Dart entiende, devolviéndosela a Riverpod.
6. Riverpod guarda esa lista de farmacias en su memoria interna.
7. Automáticamente (y de forma reactiva), la **UI** detecta que Riverpod tiene nuevos datos, por lo que se redibuja.^a sí misma mostrando finalmente los pines en el mapa para el usuario.